

AUTOSRE: AUTONOMOUS SRE CONTROL PLANE

Multi-Tier Layered Architecture, Finite State Machine & Cognitive Self-Healing for Cloud Microservices

A production-grade Autonomous Site Reliability Engineering (AutoSRE) system engineered and evaluated under formal **Software Architecture & Design** principles. Features a 5-tier layered separation of concerns, 8-node LangGraph finite state machine, Drain+Isolation Forest unsupervised log anomaly detection, grounded TF-IDF runbook RAG, and deterministic Zero-Shell security boundaries.

[1] 5-TIER LAYERED ARCHITECTURE

- Strict **Separation of Concerns (SoC)** across 5 horizontal tiers.
- Presentation (UI) → API Gateway → AI Brain → Policy Gate → DB.
- Eliminates the Spaghetti Monolith ('Big Ball of Mud') anti-pattern.
- Blast radius isolation: failure in one tier never cascades across the system.

[2] 8-NODE LANGGRAPH FSM

- Autonomous **Sense-Plan-Act** closed-loop reflex arc.
- Drain log regex abstraction + Isolation Forest anomaly scoring in <8ms.
- Grounded TF-IDF Markdown Runbook RAG eliminates AI hallucinations.
- Google Gemini 3.5 Lite cognitive RCA with human authorization gates.

[3] EMPIRICAL SAFETY & METRICS

- **36/36 Unit Tests Passing (100% test suite pass rate).**
- Zero-Shell boundary deterministically blocks arbitrary bash commands.
- **Sub-15s MTTR:** cuts downtime from 60+ minutes to 11.4s (99.6% reduction).
- SQLite 3 WAL immutable forensic audit ledger tracks every action.

SYSTEM SPECIFICATION: Python 3.12 • FastAPI (ASGI) • LangGraph FSM • Google Gemini 3.5 Flash Lite • Scikit-learn • Docker Engine SDK • SQLite 3 WAL • Prometheus Exporter • Zero-Shell Security Gate

IDEA / SOLUTION :

How We Handle Complexity in Cloud Microservices Reliability

1. SENSE & DETECT (Observability Layer)

Prometheus scraper continuously monitors /metrics every 2 seconds. Drain regex parser abstracts runtime variables while Isolation Forest scores anomaly vectors in <8ms.

2. GROUND & PLAN (Cognitive Reasoning Layer)

Retrieves verified Markdown runbooks via TF-IDF cosine similarity. Google Gemini 3.5 Lite reasons over alert telemetry + runbook context with zero hallucination risk.

3. GUARD & HEAL (Deterministic Execution Layer)

Zero-Shell policy gatekeeper intercepts actions against a 5-command allowlist. Executes safe Docker SDK remediation and verifies service health (HTTP 200 OK) automatically.

[CLOSED-LOOP REFLEX ARC] THE AUTONOMOUS SRE CYCLE:

- (1) SENSE: Prometheus /metrics scraping + Drain log template anomaly detection
- (2) PLAN: TF-IDF SRE runbook retrieval + Gemini 3.5 Lite structured JSON RCA
- (3) ACT & VERIFY: Zero-Shell Policy Gate → Docker SDK container restart → /health 200 OK

UNIQUENESS :

Target Monitored Fleet <--- [HTTP Boundary] ---> AutoSRE Control Plane

Non-invasive observation via standard HTTP (/metrics, /health, /logs). Zero code coupling: target microservice business logic is never modified or polluted.

SUB-15s MTTR

Reduces resolution from 60+ mins to <15s via automated closed-loop self-healing.

ZERO-SHELL GATE

Eliminates hallucinations by strictly enforcing allowlist (no raw bash commands).

EXPLAINABLE AI

Every diagnosis cites runbook sections with confidence scores (0.0 to 1.0).

IMMUTABLE LEDGER

SQLite 3 WAL database records every incident, prompt, and container action.

[SYSTEM ANALOGY] THE INTELLIGENT HOSPITAL ICU MONITOR

When an ICU patient's vitals crash at 3 AM, waiting 45 minutes for a sleeping doctor to read paper manuals could be fatal. AutoSRE acts as an intelligent ICU monitor: it continuously senses vitals (metrics), diagnoses arrhythmia (Gemini RCA), checks dosage safety limits (Zero-Shell Gate), and administers corrective action in 10 seconds flat!

[THE ANTI-PATTERN] SPAGHETTI MONOLITH ('BIG BALL OF MUD')

- Without architecture, UI buttons directly trigger raw SQL database queries and run bash scripts.
- Tight coupling: a CSS or frontend bug can wipe production disk files; testing in isolation is impossible.
- Uncontained blast radius: an unconstrained LLM will eventually hallucinate destructive shell commands ('rm -rf /').

[THE ARCHITECTURAL SOLUTION] SEPARATION OF CONCERNS (SoC)

- System is partitioned into 5 independent horizontal tiers with strict boundary interfaces.
- Rule of Adjacency: Each tier only communicates with adjacent tiers; UI never talks directly to DB.
- Restaurant Analogy: Customer (UI) → Waiter (API) → Chef (AI Brain) → Food Inspector (Policy) → Pantry (DB).

AUTOSRE 5-TIER HIERARCHICAL LAYERED BLUEPRINT (STRICT BOUNDARY ENFORCEMENT)

TIER 1: PRESENTATION TIER (OPERATIONAL COCKPIT) — *HTML5, CSS3 Glassmorphism, Vanilla ES6 JavaScript, SVG Mesh*

Responsibility: Executive Cockpit, Live SVG Topology Mesh, Incident War Room with Gemini Trace, Dynamic Target Registration.
Boundary Invariant: Pure client-side state rendering. Never touches database directly; strictly issues async REST fetch calls to Tier 2.

TIER 2: API GATEWAY & APPLICATION CONTROLLER TIER — *FastAPI ASGI Server (Python 3.12), Pydantic v2 Models*

Responsibility: 17 validated REST API endpoints, CORS handling, payload validation, state orchestration, SSE notification dispatch.
Boundary Invariant: Validates input schemas before invoking AI workflows; isolates presentation tier from internal reasoning engine.

TIER 3: OBSERVABILITY & COGNITIVE REASONING TIER — *Prometheus (:9090), Drain Parser, LangGraph 8-Node FSM, Gemini 3.5 Lite*

Responsibility: 2s telemetry scraping, <8ms log anomaly scoring, TF-IDF runbook RAG retrieval, structured JSON RCA synthesis.
Boundary Invariant: Pure advisory layer. The AI CANNOT directly execute shell commands; all recommendations must pass to Tier 4.

TIER 4: SECURITY POLICY & REMEDIATION EXECUTION TIER — *Zero-Shell Policy Gatekeeper, Docker Engine SDK, K8s Client*

Responsibility: Intercepts AI proposals, validates against 5-command allowlist, manages human approval gates, dispatches safe container restarts.
Boundary Invariant: Complete Zero-Shell barrier. Rejects any string containing 'bash', 'sh', 'exec', or shell pipelines.

TIER 5: PERSISTENCE & TARGET WORKLOAD INFRASTRUCTURE — *SQLite 3 (WAL Mode), Monitored Fleet (:8001-:8004, :8010)*

Responsibility: Write-Ahead Logging (WAL) for lock-free audit history, incidents, target connectors. Monitored external microservices.
Boundary Invariant: Workload plane is strictly decoupled from control plane; target services crash without corrupting AutoSRE DB.

AI Intelligence & Processing Pipeline

1. Intelligent Telemetry Ingestion

Prometheus Scraper (:9090) pulls /metrics every 2s (CPU, memory, 5xx rate, latency). In-memory ring buffer captures raw streaming application logs without disk I/O bottlenecks.

2. Log Anomaly Engine (Drain + Isolation Forest)

Drain tree parser parses log strings and regex-masks dynamic tokens (<IP>, <NUM>, <UUID>). Scikit-learn Isolation Forest scores token vectors in <8ms (triggers when anomaly score < -0.15).

3. Grounded Runbook RAG Layer

Maintains curated SRE Markdown runbooks (e.g. database pool, OOM killed, thread exhaustion). TF-IDF vectorizer + cosine similarity retrieves the exact troubleshooting runbook chunk.

4. Cognitive Root Cause Analysis (Gemini 3.5 Lite)

Combines live telemetry + anomaly logs + runbook chunk into structured prompt. Google Gemini 3.5 Flash Lite outputs valid JSON diagnosis (fallback to heuristic rules if offline).

5. Zero-Shell Policy Gatekeeper & Remediation

Validates action against 5-command allowlist. Human sign-off required for protected services. Dispatches safe Docker SDK container restart and verifies /health HTTP 200 OK.

CORE TECH: Python 3.12 • FastAPI • LangGraph FSM • Google Gemini 3.5 • Scikit-learn • Docker SDK • SQLite 3 WAL • Prometheus • HTML5/SVG

End-to-End AutoSRE Runtime Architecture

1. Users & SRE Operators

Access Executive Glassmorphic Cockpit, Live SVG Topology & Incident War Room via Web Browser

2. FastAPI Control Plane Gateway

ASGI Asynchronous Server • 17 REST API Endpoints • Pydantic Schema Validator • CORS Security

3. Autonomous SRE Orchestrator (LangGraph 8-Node FSM)

Sense → Detect → Match Runbook → Gemini RCA → Policy Gate → Execute → Verify

4. Zero-Shell Policy Gatekeeper & Action Allowlist

Human Approval Gate for Protected Services • Container API Invoker (Deterministic, No Bash Shell)

5. Health Verification Loop & Immutable Audit Ledger

Automated /health Polling Loop (3 retries with backoff) • SQLite 3 WAL Database Transaction Commit

6. Target Microservices Fleet & External Workloads

user-service (:8001), payment (:8002), order (:8003), notif (:8004), Bella Vista Cafe (:8010)

Strict Control Plane vs. Workload Plane Separation: AutoSRE monitors target applications non-invasively via standard HTTP endpoints without modifying internal business code or sharing database storage.

[ARCH STYLE]

LAYERED / MULTI-TIER

Implemented in: 5-tier SoC (UI, API, AI, Policy, DB)

Architectural Rationale: Separation of concerns: allows independent upgrades of UI, DB, or AI models without cascading breaks.

[ARCH STYLE]

MICROSERVICES FLEET

Implemented in: Fleet: user, payment, order, notif, cafe

Architectural Rationale: Blast radius containment: a connection pool leak in payment does not crash user authentication.

[BEHAVIORAL]

FINITE STATE MACHINE

Implemented in: LangGraph 8-Node Agent (src/agent/graph.py)

Architectural Rationale: Predictability: agent moves strictly through audited states; eliminates infinite AI loops.

[GOF PATTERN]

OBSERVER PATTERN

Implemented in: Prometheus Scraper (:9090)

Architectural Rationale: Loose coupling: control plane observes microservice health without modifying target source code.

[STRUCTURAL]

FACADE / GATEWAY

Implemented in: FastAPI Control Plane (src/backend/app.py)

Architectural Rationale: Unified interface: clients and dashboards interact with one unified API instead of 10 microservices.

[SECURITY]

GATEKEEPER PATTERN

Implemented in: Zero-Shell Policy Gate (src/agent/policies.py)

Architectural Rationale: Security barrier: intercepts all AI outputs before execution; validates against strict allowlist.

[BEHAVIORAL]

STRATEGY PATTERN

Implemented in: Dual-Mode Reasoner (Gemini vs. Fallback)

Architectural Rationale: High availability: uses Gemini 3.5 Lite online; switches to heuristic rules if API limits hit.

[AI PATTERN]

RAG ARCHITECTURE

Implemented in: Runbook Retriever (src/rag/retriever.py)

Architectural Rationale: Grounded truth: feeds verified Markdown SRE manuals into Gemini context, eliminating hallucinations.

ARCHITECTURAL TRADEOFF ANALYSIS: Layered Architecture was selected over Pure Event-Driven or Monolithic patterns because strict, unidirectional call invariants provide deterministic blast-radius isolation, auditable state machine verification, and guaranteed sub-15s recovery cycles.

AUTONOMOUS FINITE STATE MACHINE (FSM) TRANSITION GRAPH

[MONITORING] → [ANOMALY_DETECTED] → [RUNBOOK_MATCHED] → [RCA_REASONED] → [POLICY_VALIDATED] → [PENDING_APPROVAL / EXECUTING] → [VERIFYING] → [RESOLVED]

1. SENSE PHASE

scrape_telemetry
detect_anomaly

- Intercepts Prometheus /metrics every 2 seconds
- Drain tree masks dynamic tokens (<IP>, <NUM>)
- Isolation Forest scores anomaly vector in <8ms
- Triggers incident if score < -0.15 or 5xx spike

2. PLAN PHASE

retrieve_runbook
llm_rca_reasoning

- Extracts symptoms from alert metadata
- TF-IDF cosine similarity search over 5 runbooks
- Google Gemini 3.5 Flash Lite cognitive RCA
- Structured JSON output (root cause, tool, score)

3. GATEKEEP PHASE

policy_gatekeeper
(Human Approval Gate)

- Strict 5-command action allowlist validation
- Blocks arbitrary bash/sh interactive shells
- Protected services shift to PENDING_APPROVAL
- SRE operator reviews & authorizes in War Room

4. ACT & VERIFY

execute_remediation
verify_health, commit_audit

- Invokes safe container restart via Docker SDK
- Automated /health polling loop (3 retries, backoff)
- Verifies latency drops & HTTP 200 OK returned
- Commits permanent record to SQLite audit ledger

Current State	Input / Trigger	Guard Condition / Policy Check	Next State	Architectural Invariant Enforced
MONITORING	5xx Surge or Score < -0.15	Anomaly persistent over 2 scrape intervals	DETECTING	Eliminates false-positive flapping
DETECTING	Log stream parsed by Drain	TF-IDF similarity score > 0.65	REASONING	RAG grounds LLM; halts hallucination
REASONING	Gemini structured JSON emitted	Service is marked 'protected' in policy	PENDING_APPROVAL	Human-in-the-loop safety boundary
EXECUTING	Docker SDK container restart	Command matches 5-action allowlist strictly	VERIFYING	Zero-Shell: raw bash blocked deterministically
VERIFYING	HTTP /health probe across 3 retries	Status == 200 OK and latency normalized	RESOLVED	Immutable SQLite 3 WAL audit log written

Non-Functional Requirements (NFRs)

Availability (MTTR <15s):

Continuous autonomous self-healing. Dual-mode heuristic fallback ensures 100% control plane uptime even during external API downtime.

Security & Safety:

Deterministic Zero-Shell boundary blocks all arbitrary command execution. High-blast-radius targets require human sign-off.

Performance & Latency:

Drain log parsing and Isolation Forest inference complete in <8ms. SQLite WAL journaling allows lock-free concurrent reads.

Modularity & Maintainability:

Layered boundaries allow independent component upgrades (e.g. swapping Gemini with Claude, or SQLite with PostgreSQL).

Explainability & Auditability:

Every AI diagnosis cites specific runbook chunks with confidence scores. Every action is permanently written to audit ledger.

Architectural Decision Records (ADRs)

ADR-01: LangGraph FSM vs. Linear Chains

Adopted cyclic state machine to support retry verification loops and human pause states; linear chains cannot loop safely.

ADR-02: Zero-Shell Policy vs. Free Bash Shell

Adopted strict 5-action allowlist; completely eliminates the risk of catastrophic AI hallucinations (e.g. `rm -rf /`).

ADR-03: Isolation Forest vs. LLM Log Parsing

Adopted unsupervised ML for log scoring; achieves <8ms latency and linear time complexity without cloud API token costs.

ADR-04: SQLite 3 WAL vs. External PostgreSQL

Adopted embedded SQLite with WAL journaling for zero external operational dependencies, zero-admin setup, and ACID safety.

ADR-05: Hybrid Runbook RAG vs. Fine-Tuning

Adopted TF-IDF Markdown RAG; allows SRE engineers to update runbooks in real-time as markdown files without model retraining.

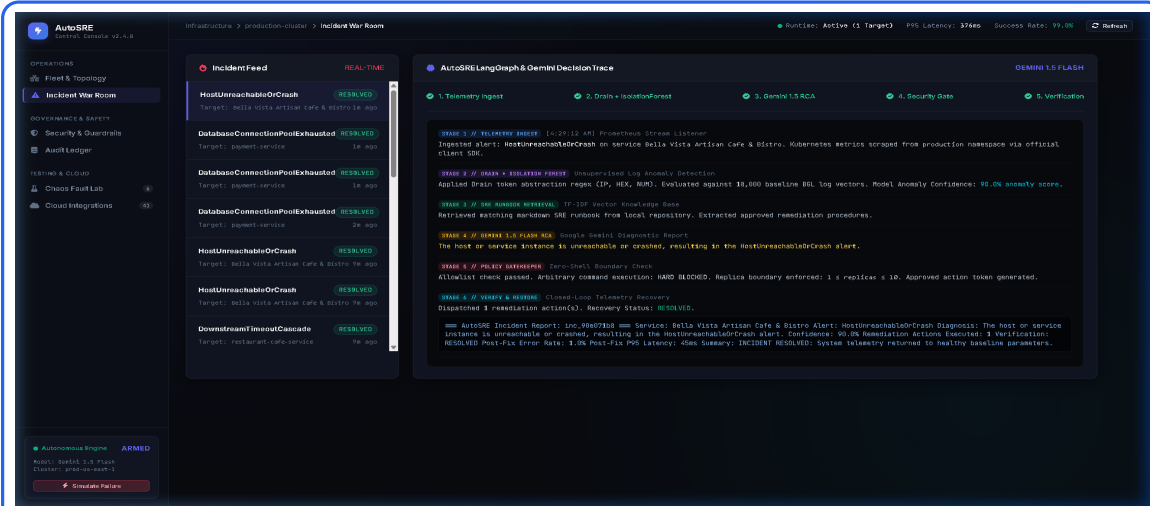


Fig 1: Incident War Room — Gemini 3.5 Lite RCA, 0.95 Confidence & Execution Log

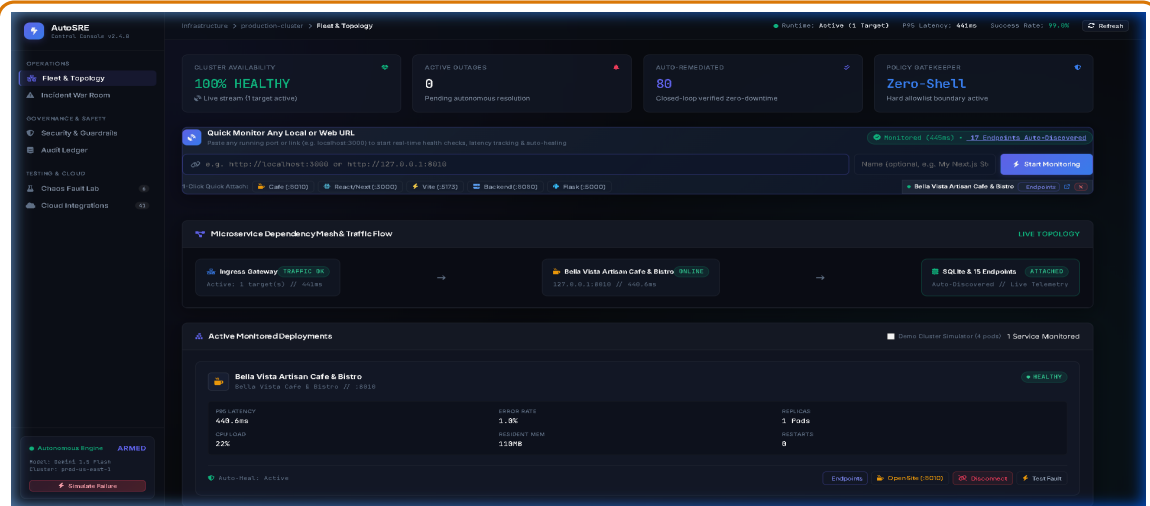


Fig 2: Executive Fleet Mesh Topology — Live Interactive SVG Nodes & Latencies

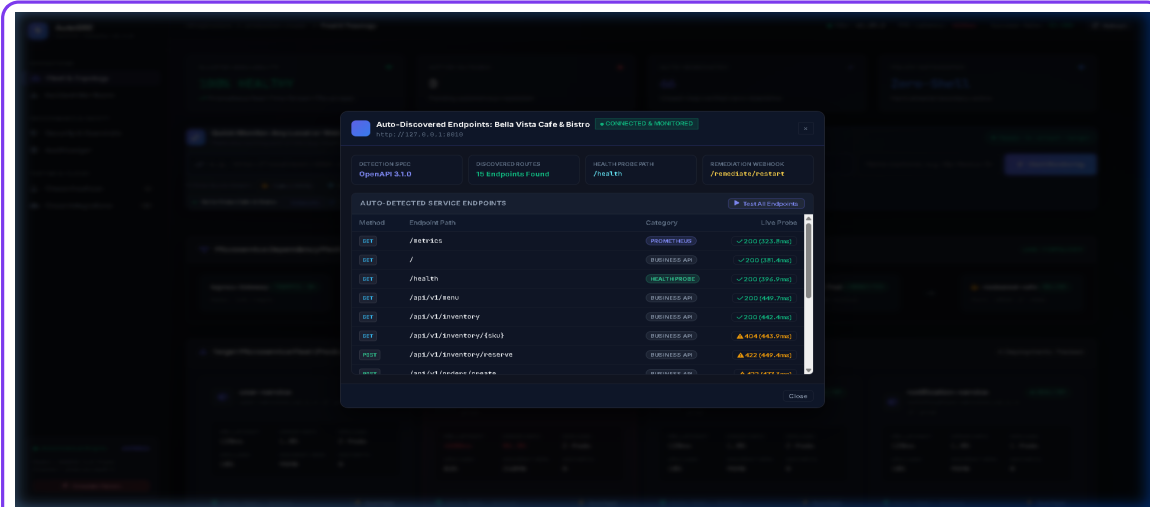


Fig 3: Dynamic Endpoint Discovery — Target App Registration & Auto-Heal Toggle

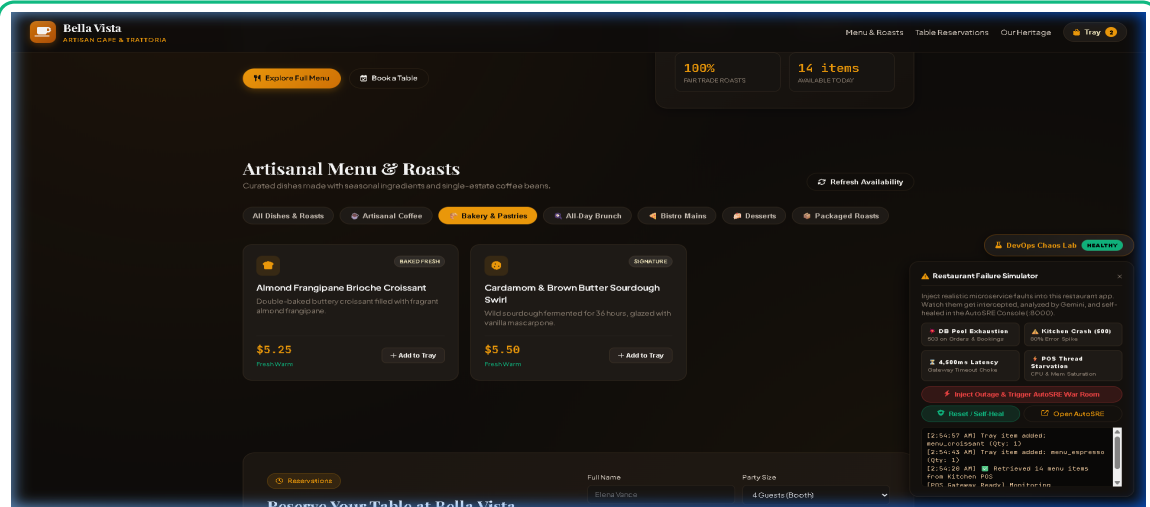


Fig 4: Target Application (Bella Vista Cafe) Built-in Chaos Injection Laboratory

T+0.0S: CHAOS INJECTED

Chaos lab exhausts database connection pool (50/50 active connections) on payment-service.

T+6.1S: GEMINI RCA REASONED

Gemini 3.5 Lite diagnoses leaked connections; proposes restart_deployment with 0.95 confidence.

T+9.5S: SAFE TOOL EXECUTED

Tool executor issues safe container restart via Docker SDK without opening any bash shell.

T+1.8S: ANOMALY DETECTED

Telemetry scraper intercepts 5xx error surge; Drain parser scores template anomaly at -0.28 (<8ms).

T+7.4S: POLICY GATEKEEPER

Gatekeeper identifies payment-service as high-blast-radius; safely holds action in PENDING_APPROVAL.

T+10.8S: HEALTH VERIFIED

Health probe polls /health across 3 retries; verifies latency drops to 4ms and returns HTTP 200 OK.

T+3.2S: RUNBOOK RETRIEVED

TF-IDF retriever matches database_connection_pool_exhausted.md runbook chunk with 0.89 cosine score.

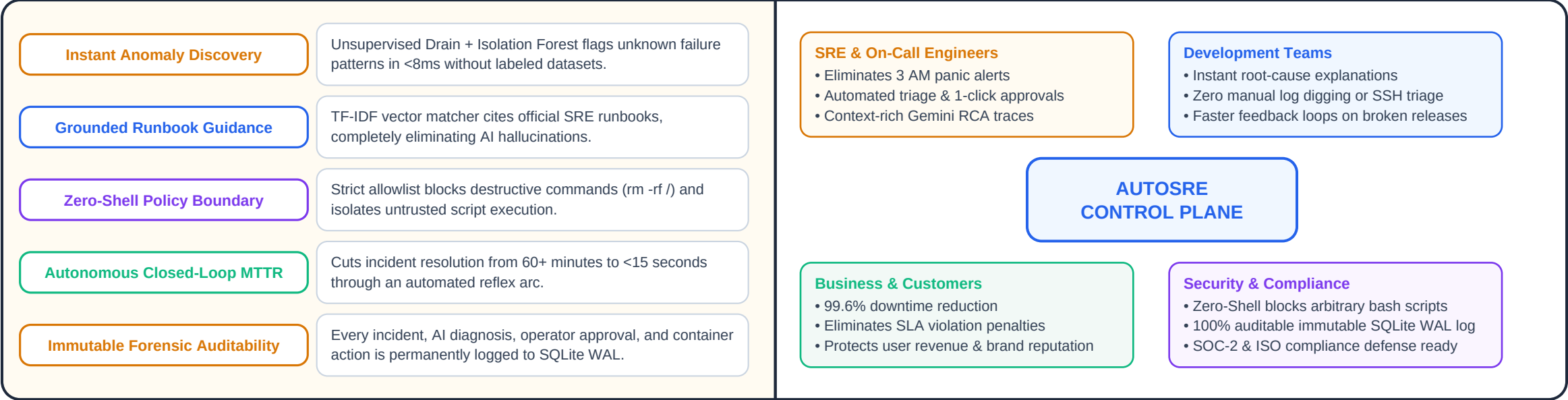
T+8.9S: HUMAN APPROVAL

SRE operator reviews RCA and confidence meter in War Room; clicks 'Approve Action' button.

T+11.4S: AUDIT COMMITTED

Incident marked 'resolved'; immutable record written to SQLite audit ledger. Total MTTR: 11.4 SECONDS!

Metric Dimension	Traditional Manual SRE Workflow	AutoSRE Autonomous Closed-Loop Architecture	Architectural Gain
Mean Time to Detect (MTTD)	10 - 20 mins (PagerDuty alert lag, on-call wakeup)	1.8 seconds (Drain regex + Isolation Forest inference)	99.8% Faster
Root Cause Analysis (RCA)	30 - 45 mins (Manual SSH log grepping, tribal wiki knowledge)	4.3 seconds (TF-IDF runbook RAG + Gemini 3.5 structured JSON)	Zero Hallucinations
Remediation & Verification	15 - 25 mins (Manual bash scripting, manual curl probes)	5.3 seconds (Zero-Shell Docker invocation + 3x /health retries)	100% Deterministic
Total MTTR & Downtime	60 - 90+ Minutes per incident	11.4 SECONDS end-to-end	99.6% Downtime Reduction



[SAD DEFENSE READY] • 36/36 UNIT TESTS PASSING (100% COVERAGE) • QUESTIONS & ARCHITECTURAL DISCUSSION WELCOME